Comparing Forecasters for Serverless Invocation Provisioning

Ehsan Moradi
Department of Computer Engineering
Amirkabir University of Technology
Tehran, Iran
your.email@aut.ac.ir

Abstract—Serverless platforms keep warm containers ready for a changing arrival rate. Too few containers cause cold starts, and too many sit idle. This paper compares six forecasters of per-minute function invocations on a public Azure Functions trace: three simple baselines, exponential smoothing, a pretrained time-series model, and a small LSTM. Functions are grouped into sparse, daily, bursty, and other workloads. Each forecast covers the next 60 minutes. Error is reported with a provisioning cost that counts missed invocations and extra container-minutes. On a typical function, a seasonal naive forecast has the lowest error. A high percentile of recent load misses fewer invocations and holds more idle capacity. Choosing one of these two methods per class on two days, then applying that choice on the next two days, preserves the same trade-off. The learned models do not beat the simple methods under this protocol.

Index Terms—serverless computing, workload forecasting, cold start, provisioning

I. Introduction

A serverless platform receives a changing number of function invocations and must decide how many warm containers to keep. A shortfall makes some requests wait for a cold start. Spare containers consume memory and money while they sit idle. A forecast of the next hour can drive that decision, but a lower forecast error does not by itself mean a better operating point. Under-predicting and over-predicting have different costs.

Forecasting is already part of production resource management. The public Azure Functions study uses a time-series forecast inside a larger keep-alive policy [?]. What is still easy to miss is that functions are not one workload. A timer with a daily shape, a rarely called HTTP function, and a bursty queue consumer can prefer different forecasters. Average error over the whole trace hides that split.

This paper is a comparative study, not a new scheduler. On 80 functions from the 2019 Azure Functions invocation trace, it asks which forecaster an operator should use for which class of function, and what that choice costs in missed invocations versus idle containers. The contributions are a class-aware comparison, a provisioning score beside forecast error, and a selection rule fit on two days and checked on the next two.

II. Related Work

Shahrad et al. characterize the Azure Functions workload and show that most functions are invoked rarely [?]. Their hybrid policy keeps containers warm from a histogram of idle times and falls back to a time-series forecast when the histogram does not cover the idle gaps. This paper uses the same public invocation counts. It does not reimplement that keep-alive policy. The decision here is the number of containers implied by a forecast of arrivals, under a fixed service rate.

Chronos trains a probabilistic forecaster on a large collection of time series and can forecast a new series without retraining [?]. Chronos-Bolt is a faster variant that emits quantile forecasts for the whole horizon in one pass. We use the small Chronos-Bolt checkpoint with no fine-tuning, and we score its median and two upper quantiles. Upper quantiles are a direct way to buy fewer shortfalls by preparing more containers.

Learned forecasters, including recurrent networks, have been trained to pre-warm serverless containers. The question here is narrower. Under one public trace, one horizon, and one capacity assumption, do exponential smoothing, Chronos, or an LSTM beat simple rules that an operator can run without training?

III. Setup

The trace records invocations per minute for 14 days [?]. Class statistics use days 1–10 only. A missing day is treated as 1,440 zeros. A function is sparse when at least 90% of those minutes are zero. Otherwise it is daily when the autocorrelation at a lag of one day exceeds 0.30. A remaining function is bursty when its coefficient of variation is at least 2.05, the 75th percentile among non-sparse functions. The rest are labeled other. We draw 20 eligible functions from each class with seed 42. A function is eligible when it is invoked at least once in days 1–10 and at least once in days 11–14. The trace contains 68,394 functions in the first ten days, of which 54,502 are sparse.

Each forecast uses the previous 360 minutes and predicts the next 60. A new forecast starts every 60 minutes across days 11–14, so the 96 windows cover those four days without overlap. Days 1–10 are history. Days 11–12

xxxx-..-….-….-x/xx/$xx.00 ©2026IEEE

are used only to choose the class rule. Days 13–14 are the held-out check.

Six point forecasts are compared. Seasonal naive repeats the same hour of the previous day when the training day-lag autocorrelation exceeds 0.30, and otherwise repeats the previous 60 minutes. Last-value repeats the latest observed minute. The trailing percentile holds the 95th percentile of the previous 10 minutes for the whole hour. ETS is damped additive Holt smoothing fitted to the same 360-minute context, with no seasonal term. Chronos is the median forecast of amazon/chronos-bolt-small, applied with no training on this trace. The LSTM is one shared network with a single layer of 32 units. It is trained on windows from days 1–8, early-stopped on days 9–10, and run for 20 epochs. Each LSTM window is scaled by the mean of its own context and scaled back after prediction.

The provisioning score assumes that one warm container serves 10 invocations in a minute. For a forecast $\hat{y}_t$ of the actual count y_t,

$$U = \sum_t \max(0,\, y_t - \hat{y}_t), \tag{1}$$

$$O = \sum_t \max(0,\, \lceil \hat{y}_t/10 \rceil - \lceil y_t/10 \rceil)\,. \tag{2}$$

U counts missed invocations, a proxy for cold-start risk. O counts extra container-minutes, a proxy for waste. Forecast error is the symmetric mean absolute percentage error (sMAPE). A minute where both the actual value and the forecast are zero contributes nothing. On days 11–12, one bursty function accounts for 98.7% of that class's missed invocations under last-value, so class means follow a single series. Every table below reports the median across functions.

IV. Evaluation

Table I summarizes the four test days. Seasonal naive has the lowest overall median sMAPE, 7.1%. The gap is largest on daily functions, where its median sMAPE is 3.9%, against 37.8% for last-value and 134.4% for the trailing percentile. Fig. 1 shows why. On the daily example, yesterday's shape follows the load. The trailing percentile sits in steps above it. On the sparse example, arrivals are isolated spikes. The LSTM has the lowest sparse sMAPE, 0.7%, and almost no idle containers, because it predicts low. That same behavior misses load on the other classes: its overall median shortfall is 1,352 invocations.

The trailing percentile has the smallest overall shortfall among the point forecasts, 95.5 missed invocations, and the seasonal naive forecast misses 206. The percentile pays for that reduction with 729 extra container-minutes, against 80.5 for seasonal naive. ETS and Chronos land at the expensive end of Fig. 2. Raising the Chronos forecast from the median to the 90th percentile cuts the median shortfall from 778.5 to 99.4 and raises extra container-minutes from 3,582 to 4,833.5. The 90th percentile meets the trailing percentile on shortfall and still wastes more.

The selection rule uses only point forecasts. On days 11–12, each class takes the forecast with the lowest median shortfall, then the lower waste, then the lower sMAPE. Sparse functions select seasonal naive. Daily, bursty, and other functions select the trailing percentile. Table II applies that frozen rule to days 13–14. Across all functions the rule misses 16.6 invocations at the median, against 40.0 for always using seasonal naive and 21.0 for always using the trailing percentile. Its sMAPE, 12.4%, sits between those two policies, and its waste, 113.5 container-minutes, stays close to the trailing percentile. The rule therefore buys the shortfall of the high forecast without giving up the seasonal forecast on sparse functions. It does not dominate on error or on waste.

The study has four limits. The trace is from 2019 and contains 80 sampled functions, not a live deployment. The capacity of 10 invocations per container-minute is an assumption, not a measured service rate. U and O are computed from the trace, not from cold starts on a platform such as OpenWhisk. Medians are the right summary here, and they are not a substitute for a cost model that weights a few very large functions the way a provider bill would.

V. Conclusion

On this trace, the useful choice is between two simple forecasts. Seasonal naive is the accurate one, especially for functions with a daily shape. The trailing percentile misses fewer invocations because it keeps spare containers. A rule that picks seasonal naive for sparse functions and the trailing percentile otherwise keeps that trade-off on held-out days. Exponential smoothing, Chronos-Bolt, and the LSTM do not improve on those two methods under a 360-minute context and a fixed container capacity. A next measurement is to replace the capacity assumption with cold starts recorded on a real platform.

TABLE I
Median sMAPE, missed invocations, and extra container-minutes on days 11–14.

Model	Metric	Sparse	Daily	Bursty	Other	All
Seasonal naive	sMAPE (%)	0.9	3.9	41.0	12.9	7.1
	Missed	23.0	122.5	1170.5	259.5	206.0
	Extra	13.0	43.0	719.0	150.5	80.5
Last value	sMAPE (%)	0.9	37.8	48.2	11.9	19.2
	Missed	27.5	1152.5	1151.0	259.0	916.0
	Extra	0.0	31.0	593.5	93.5	63.5
Trailing percentile	sMAPE (%)	3.8	134.4	74.6	12.9	24.7
	Missed	27.1	24.6	554.1	115.4	95.5
	Extra	87.5	4386.5	2183.0	98.0	729.0
ETS	sMAPE (%)	46.9	183.8	149.1	10.5	71.5
	Missed	33.0	953.1	1084.3	214.6	823.8
	Extra	1325.5	4582.5	3751.5	182.5	2765.0
LSTM	sMAPE (%)	0.7	40.0	39.1	48.4	39.9
	Missed	34.0	1540.4	1689.5	1734.8	1352.0
	Extra	0.0	0.0	196.0	122.0	3.5
Chronos	sMAPE (%)	198.7	141.3	159.8	12.0	143.3
	Missed	34.0	1146.5	1438.4	146.6	778.5
	Extra	5705.0	2955.0	3864.0	433.0	3582.0

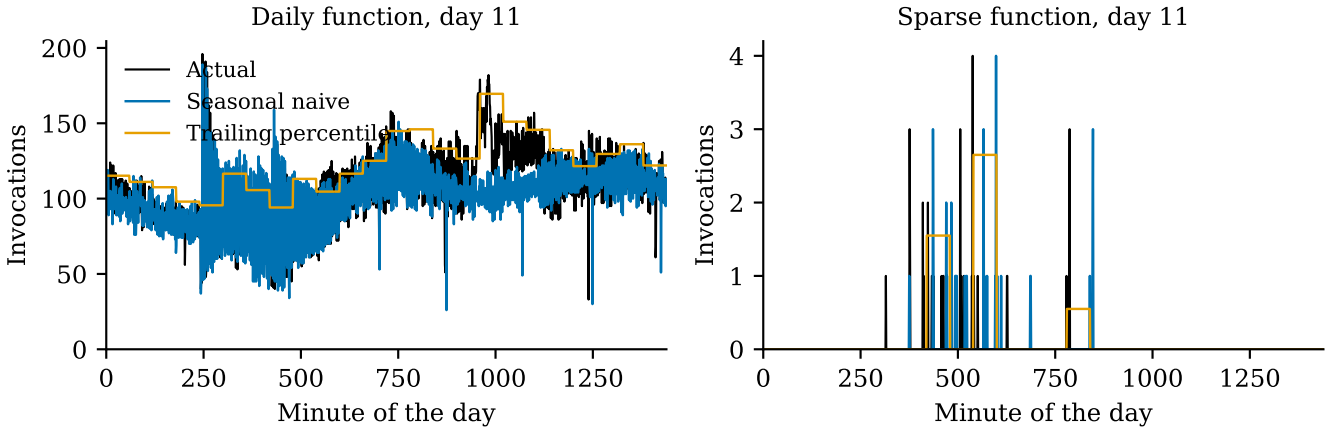


Fig. 1. Day 11 for one daily function and one sparse function. Seasonal naive follows the daily shape. The trailing percentile stays above that shape and reacts at the sparse spikes.

TABLE II
Held-out medians on days 13–14. The rule uses seasonal naive for sparse functions and the trailing percentile otherwise.

Policy	sMAPE (%)	Missed	Extra
Selection rule	12.4	16.6	113.5
Always seasonal naive	2.7	40.0	15.0
Always trailing percentile	16.2	21.0	120.5

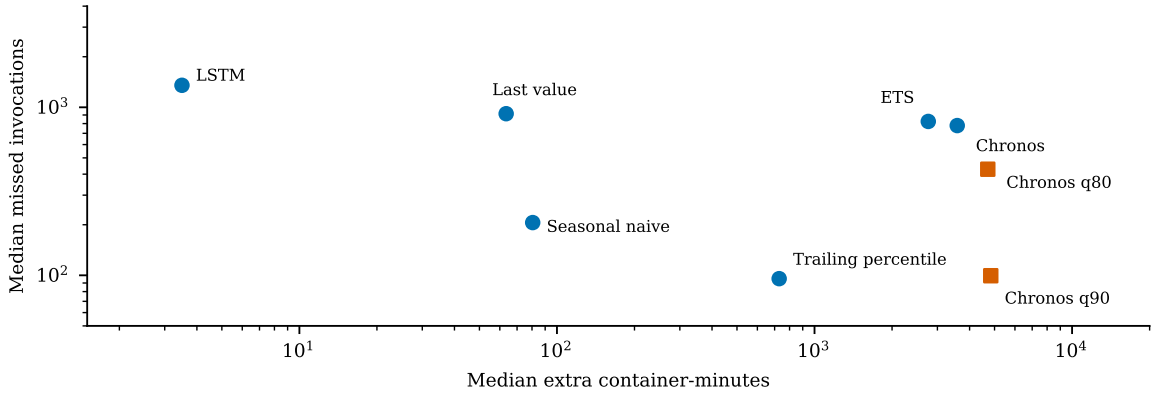


Fig. 2. Median missed invocations against median extra container-minutes on days 11–14. Both axes are logarithmic. Orange squares are Chronos forecasts at the 80th and 90th percentiles. Closer to the lower left is better.